

Gmapping-SLAM mapping

This lesson uses the Raspberry Pi 5 as an example. For Raspberry Pi and Jetson Nano boards, you need to open a terminal on the host computer and enter the command to enter the Docker container. Once inside the Docker container, enter the commands mentioned in this lesson in the terminal. For instructions on entering the Docker container from the host computer, refer to **[01. Robot Configuration and Operation Guide]** -- **[5.Enter Docker (For JETSON Nano and RPi 5)]**.

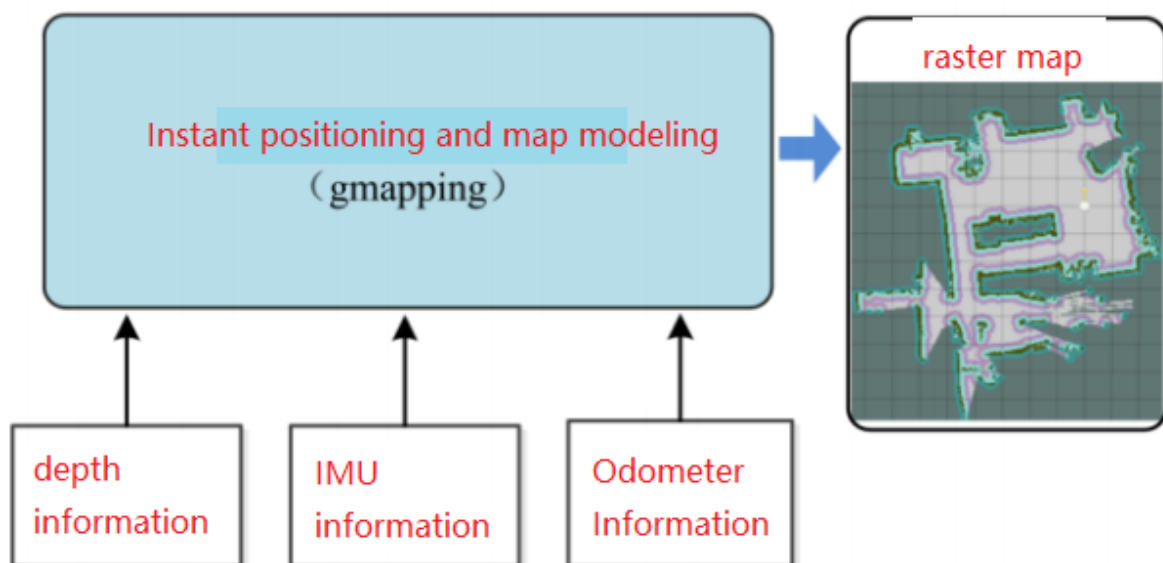
For Orin and RDK boards, simply open the terminal and enter the commands mentioned in this lesson.

1. Algorithm Introduction

- gmapping only works when the number of 2D laser points in a single frame is less than 1440. If the number of laser points in a single frame is greater than 1440, the error [[mapping-4] process has died] will occur.
- gmapping is a commonly used open-source SLAM algorithm based on the filter SLAM framework.
- gmapping is based on the RBPF particle filter algorithm, separating the real-time localization and mapping processes, performing localization first and then mapping. Gmapping makes two major improvements to the RBPF algorithm: improved proposal distribution and selective resampling.

Advantages: Gmapping can build indoor maps in real time. It requires less computation and offers high accuracy for small scene maps.

Disadvantages: As the scene size increases, more particles are required. Because each particle carries a map, the memory and computational overhead required for building large maps increases. Therefore, it is not suitable for building large scene maps. Furthermore, it lacks loop detection, which can cause map misalignment when loops are closed. While increasing the number of particles can close the map, it comes at the expense of increased computation and memory.



2. Program Functionality

After running the program, the map building interface will appear in rviz. Use the keyboard or gamepad to control the robot until the map is built. Then, run the save map command to save the map. **(For the Ackermann model car, if you want better mapping, try adding some stopping motion when turning, and avoid making a sharp turn.)**

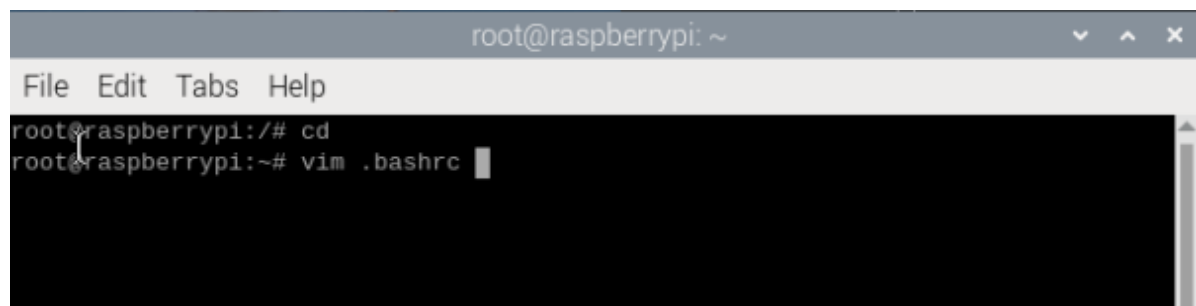
3. Pre-use Configuration

This vehicle is equipped with a USB camera, a depth camera, and two different models of LiDAR. However, because it cannot automatically recognize the products, the machine type and LiDAR model need to be manually set.

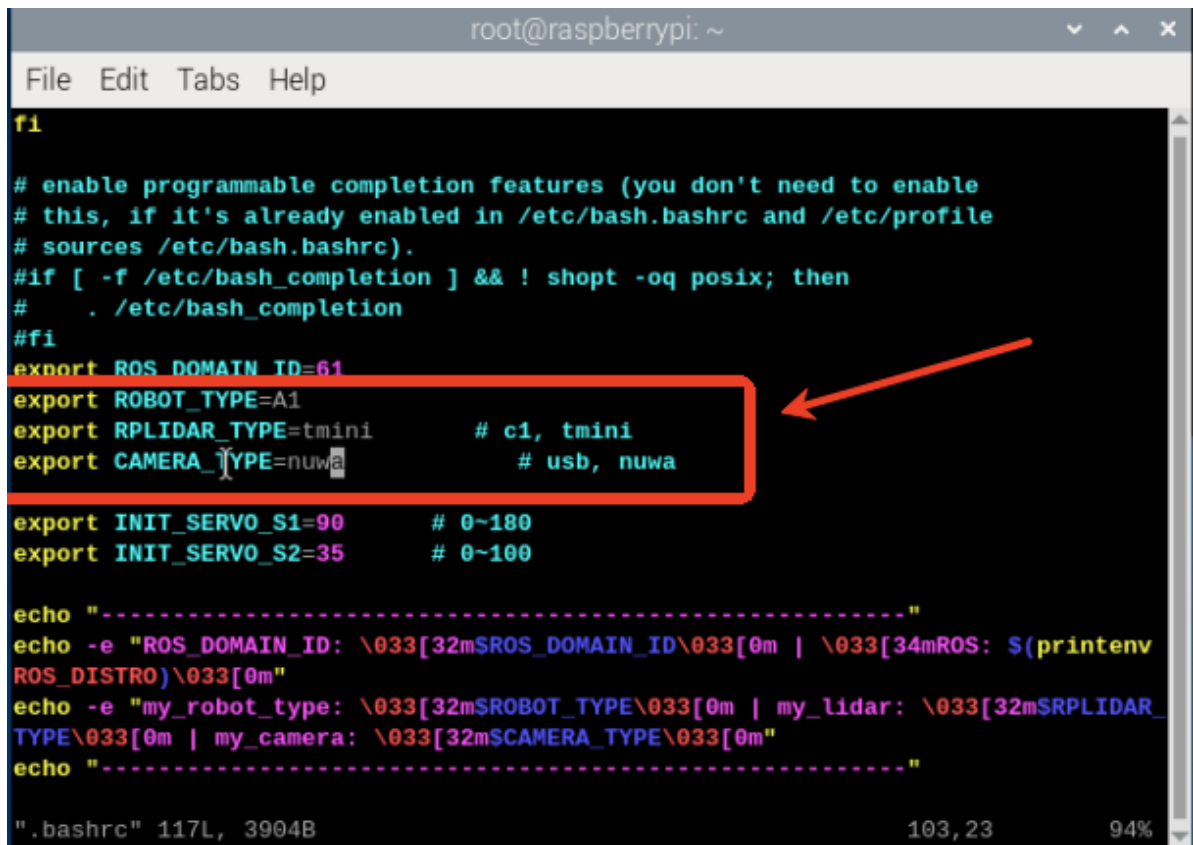
For PI5/JETSON NANO controllers, you need to enter a Docker container first; for RDK X5 and Orin boards, this is not required.

Based on the vehicle model, LiDAR type, and camera type, the following modifications should be made.

```
root@ubuntu:/# cd
root@ubuntu:~# vim .bashrc
```



Find this location and press i on your keyboard to change the camera and LiDAR models to the corresponding ones. The default values are Tmini and Nuwa.



```
root@raspberrypi: ~
File Edit Tabs Help

f1
# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
#if [ -f /etc/bash_completion ] && ! shopt -oq posix; then
#   . /etc/bash_completion
#fi
export ROS_DOMAIN_ID=61
export ROBOT_TYPE=A1
export RPLIDAR_TYPE=tmini      # c1, tmini
export CAMERA_TYPE=nuwa       # usb, nuwa

export INIT_SERVO_S1=90      # 0~180
export INIT_SERVO_S2=35      # 0~100

echo "-----"
echo -e "ROS_DOMAIN_ID: \033[32m$ROS_DOMAIN_ID\033[0m | \033[34mROS: $(printenv
ROS_DISTRO)\033[0m"
echo -e "my_robot_type: \033[32m$ROBOT_TYPE\033[0m | my_lidar: \033[32m$RPLIDAR_
TYPE\033[0m | my_camera: \033[32m$CAMERA_TYPE\033[0m"
echo "-----"

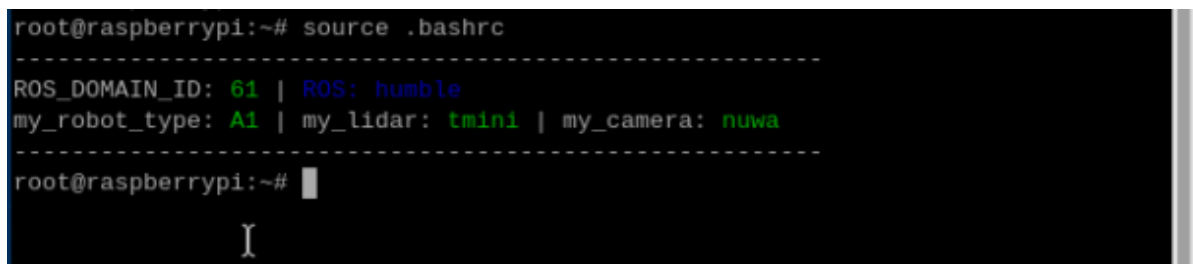
".bashrc" 117L, 3904B                                103,23          94%
```

After making the changes, save and exit vim, then execute:

```
root@raspberrypi:~# source .bashrc
```

```
-----
ROS_DOMAIN_ID: 61 | ROS: humble
my_robot_type: A1 | my_lidar: tmini | my_camera: nuwa
-----
```

```
root@raspberrypi:~#
```



```
root@raspberrypi:~# source .bashrc
-----
ROS_DOMAIN_ID: 61 | ROS: humble
my_robot_type: A1 | my_lidar: tmini | my_camera: nuwa
-----
root@raspberrypi:~#
```

4. Program Startup

4.1. Startup Commands

For PI5/JETSON NANO controllers, you need to enter a Docker container first; for RDK X5 and Orin boards, this is not required.

Entering a Docker container (see the Docker course for steps: 4. Docker Startup Script).

All commands below must be executed within the same Docker container (see the Docker course for steps: 3. Docker Commit and Multi-Terminal Entry).

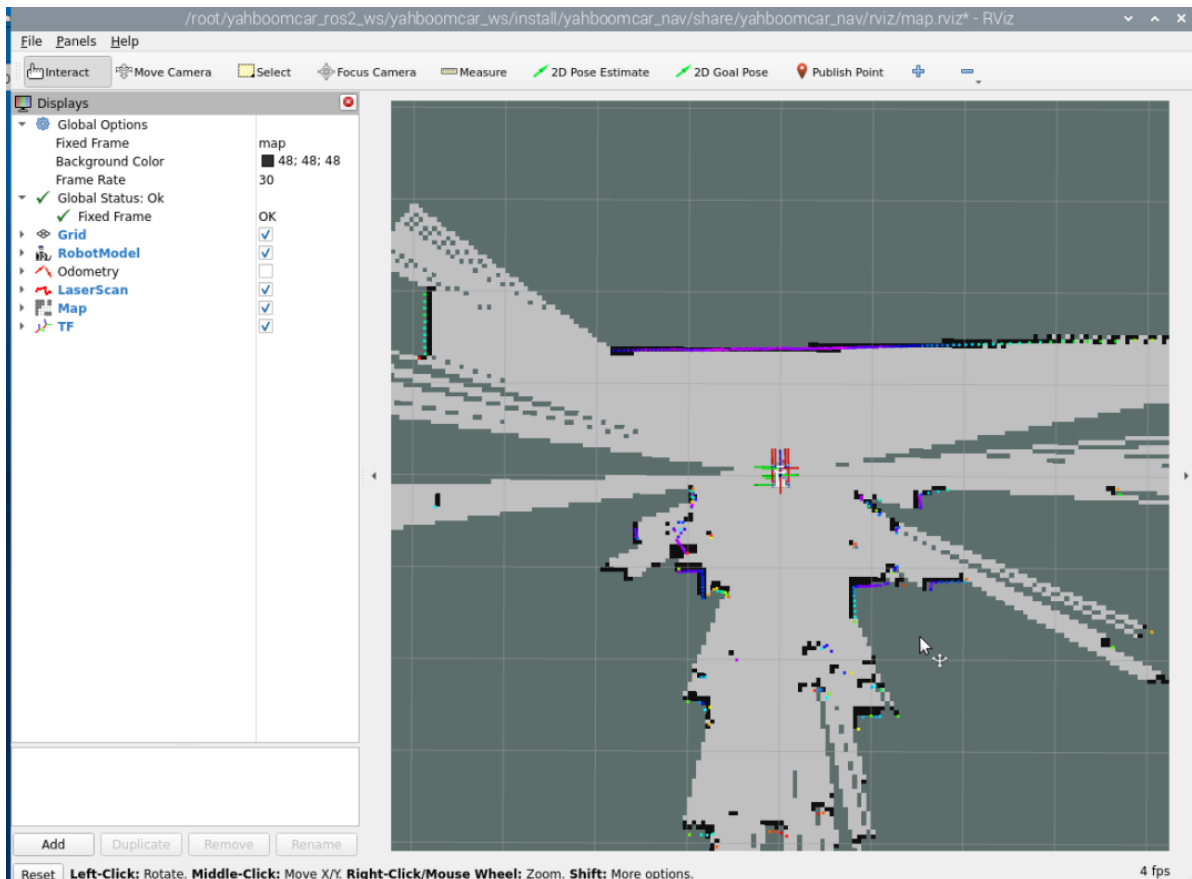
1. Start Mapping

```
ros2 launch yahboomcar_nav map_gmapping_launch.py
```

```
root@raspberrypi: ~
File Edit Tabs Help
[slam_gmapping-13] -maxUrange 5 -maxUrange 8 -sigma 0.02 -kernelSize 1 -lstep 0.05 -lobsGain 3 -astep 0.05
[slam_gmapping-13] -srr 0.1 -srt 0.2 -str 0.1 -stt 0.2
[slam_gmapping-13] -linearUpdate 1 -angularUpdate 0.5 -resampleThreshold 0.5
[slam_gmapping-13] -xmin -10 -xmax 10 -ymin -10 -ymax 10 -delta 0.05 -particles 50
[slam_gmapping-13] [INFO] [1754986373.663243149] [slam_gmapping]: Initialization complete
[slam_gmapping-13] Laser Pose= 0.0060585 -5.54004e-08 3.14158
[slam_gmapping-13] update frame 0
[slam_gmapping-13] update ld=0 ad=0
[slam_gmapping-13] m_count 0
[slam_gmapping-13] Registering First Scan
[slam_gmapping-13] update frame 15
[slam_gmapping-13] update ld=4.11275e-06 ad=0.000678839
[slam_gmapping-13] m_count 1
[slam_gmapping-13] Laser Pose= 0.0060585 -1.19172e-06 3.1414
[slam_gmapping-13] Average Scan Matching Score=314.709
[slam_gmapping-13] neff= 50
[slam_gmapping-13] Registering Scans:Done
[slam_gmapping-13] update frame 35
[slam_gmapping-13] update ld=4.78913e-06 ad=0.000790481
[slam_gmapping-13] m_count 2
[slam_gmapping-13] Laser Pose= 0.0060585 -1.67923e-06 3.14132
[slam_gmapping-13] Average Scan Matching Score=314.792
[slam_gmapping-13] neff= 50
[slam_gmapping-13] Registering Scans:Done
```

Enter the command to start rviz visualization of the map.

```
ros2 launch yahboomcar_nav display_map_launch.py
```



The program has controller control enabled by default. If you're using a controller, you can now connect it directly to the receiver for control. If you prefer keyboard control, enter:

```
#keyboard
ros2 run yahboomcar_ctrl yahboom_keyboard
```

```
root@raspberrypi: ~
File Edit Tabs Help
root@raspberrypi:~# ^C
root@raspberrypi:~# ^C
root@raspberrypi:~# ^C
root@raspberrypi:~# ros2 run yahboomcar_ctrl yahboom_keyboard

Control Your SLAM-Not!
-----
Moving around:
  u      i      o
  j      k      l
  m      ,      .

q/z : increase/decrease max speeds by 10%
w/x : increase/decrease only linear speed by 10%
e/c : increase/decrease only angular speed by 10%
t/T : x and y speed switch
s/S : stop keyboard control
space key, k : force stop
anything else : stop smoothly

CTRL-C to quit

currently:      speed 0.2      turn 1.0
```

Then control the car to slowly navigate the area you want to map. After mapping is complete, enter the following command to save the map. Enter:

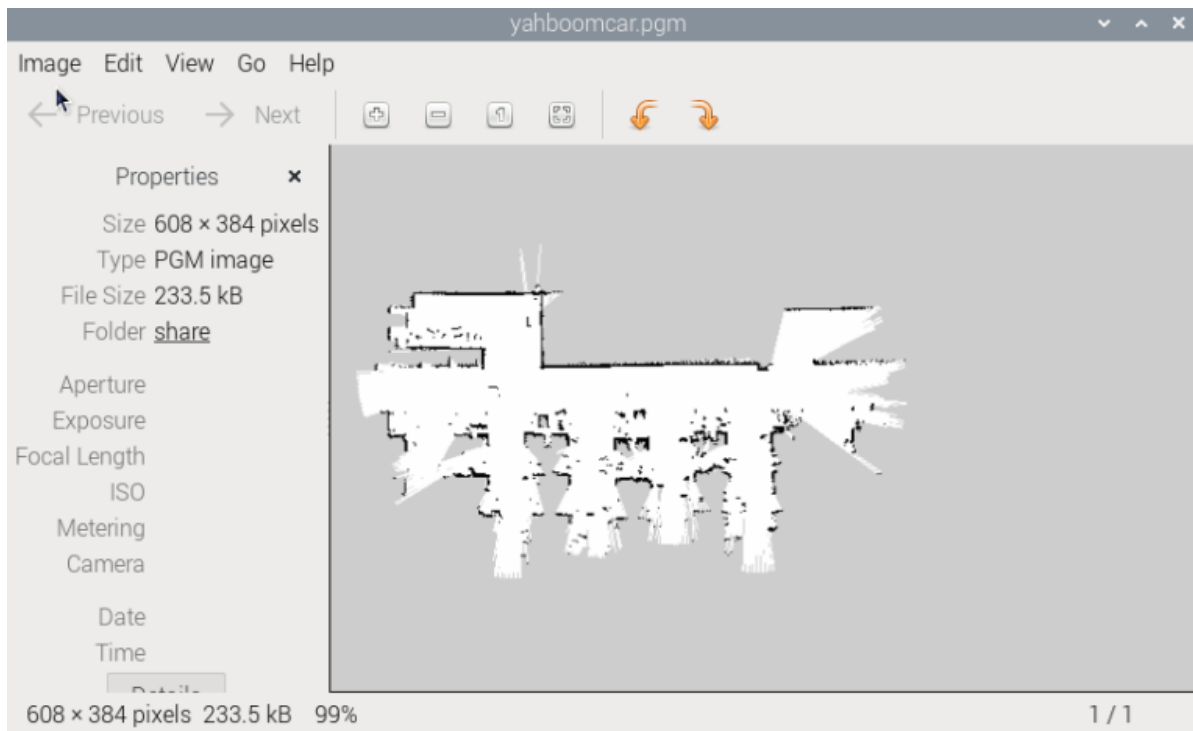
```
ros2 launch yahboomcar_nav save_map_launch.py
```

```
root@raspberrypi:~# ros2 launch yahboomcar_nav save_map_launch.py
[INFO] [launch]: All log files can be found below /root/.ros/log/2025-08-12-16-24-21-501097-raspberrypi-37294
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [map_saver_cli-1]: process started with pid [37298]
[map_saver_cli-1] [INFO] [1754987062.189949739] [map_saver]:
map_saver lifecycle node launched.
[map_saver_cli-1]      Waiting on external lifecycle transitions to activate
[map_saver_cli-1]      See https://design.ros2.org/articles/node_lifecycle.html for more information.
[map_saver_cli-1] [INFO] [1754987062.190127090] [map_saver]: Creating
[map_saver_cli-1] [INFO] [1754987062.190683088] [map_saver]: Configuring
[map_saver_cli-1] [INFO] [1754987062.199681739] [map_saver]: Saving map from 'map' topic to '/root/yahboomcar_
ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/yahboomcar' file
[map_saver_cli-1] [WARN] [1754987062.199737832] [map_saver]: Free threshold unspecified. Setting it to default
value: 0.250000
[map_saver_cli-1] [WARN] [1754987062.199756813] [map_saver]: Occupied threshold unspecified. Setting it to default
value: 0.650000
[map_saver_cli-1] [WARN] [1754987062.238155373] [map_io]: Image format unspecified. Setting it to: pgm
[map_saver_cli-1] [INFO] [1754987062.241329417] [map_io]: Received a 384 X 608 map @ 0.05 m/pix
[map_saver_cli-1] [INFO] [1754987062.378335442] [map_io]: Writing map occupancy data to /root/yahboomcar_ros2_
_ws/yahboomcar_ws/src/yahboomcar_nav/maps/yahboomcar.pgm
[map_saver_cli-1] [INFO] [1754987062.409924713] [map_io]: Writing map metadata to /root/yahboomcar_ros2_ws/ya
hboomcar_ws/src/yahboomcar_nav/maps/yahboomcar.yaml
[map_saver_cli-1] [INFO] [1754987062.410375119] [map_io]: Map saved
[map_saver_cli-1] [INFO] [1754987062.410407026] [map_saver]: Map saved successfully
[map_saver_cli-1] [INFO] [1754987062.416145153] [map_saver]: Destroying
[INFO] [map_saver_cli-1]: process has finished cleanly [pid 37298]
```

A map named yahboomcar will be saved. This map is saved in:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/yahboomcar.yaml
```

Two files will be generated, one named yahboomcar.pgm. The map is as follows. (For Orin boards, you can directly double-click the map. For Raspberry Pi boards, you need to copy the file to /root/share in Docker.) (This is the shared folder, then view it on the host machine.)



yahboomcar.yaml, take a look at the contents of the yaml.

```
image: yahboomcar.pgm
mode: trinary
resolution: 0.05
origin: [-10, -21.2, 0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.25
```

- image: The image representing the map, i.e., yahboomcar.pgm
- mode: This property can be one of trinary, scale, or raw, depending on the selected mode. Trinary is the default mode.
- resolution: The resolution of the map, in meters/pixels
- origin: The 2D coordinate system at the bottom left corner of the map. Pose (x, y, yaw), where yaw is counterclockwise rotation (yaw = 0 means no rotation). Currently, many parts of the system ignore the yaw value.
- negate: Whether to invert the meaning of white/black and free/occupied (this does not affect the interpretation of thresholds).
- occupied_thresh: Pixels with an occupied probability greater than this threshold are considered fully occupied.
- free_thresh: Pixels with an occupied probability less than this threshold are considered completely free.

5. View the Node Communication Graph

In the terminal, enter:

```
ros2 run rqt_graph rqt_graph
```



```

        [os.path.join(get_package_share_directory('yahboomcar_nav'), 'launch'),
          '/laser_bringup_launch.py']]
    )

    slam_gmapping_launch =
    IncludeLaunchDescription(PythonLaunchDescriptionSource(
        [os.path.join(get_package_share_directory('slam_gmapping'), 'launch'),
          '/slam_gmapping.launch.py'])
    )

    return LaunchDescription([laser_bringup_launch, slam_gmapping_launch])

```

Here a launch file - slam_gmapping_launch is started. See slam_gmapping_launch in detail. The file is located at

```

~/yahboomcar_ros2_ws/software/library_ws/src/ros2_gmapping/slam_gmapping/launch/
slam_gmapping.launch.py

```

slam_gmapping.launch.py,

```

from launch import LaunchDescription
from launch.substitutions import EnvironmentVariable
import launch.actions
import launch_ros.actions
import os
from ament_index_python.packages import get_package_share_directory

def generate_launch_description():

    return LaunchDescription([
        launch_ros.actions.Node(
            package='slam_gmapping',
            executable='slam_gmapping',
            output='screen',
            parameters=
            [os.path.join(get_package_share_directory("slam_gmapping"), "params",
            "slam_gmapping.yaml")]),
    ])

```

Here the slam_gmapping node is started and the slam_gmapping.yaml parameter file is loaded. The file is located at

```

~/yahboomcar_ros2_ws/software/library_ws/src/ros2_gmapping/slam_gmapping/params/
slam_gmapping.yaml

```

slam_gmapping.yaml

```

/slam_gmapping:
  ros__parameters:
    angularUpdate: 0.5
    astep: 0.05
    base_frame: base_footprint
    map_frame: map

```



```
odom_frame: odom
delta: 0.05
iterations: 4
kernelSize: 1
lasamplerange: 0.005
lasamplestep: 0.005
linearUpdate: 1.0
llsamplerange: 0.01
llsamplestep: 0.01
lsigma: 0.075
lskip: 0
lstep: 0.05
map_update_interval: 5.0
maxRange: 8.0
maxUrange: 5.0
minimum_score: 0.1
occ_thresh: 0.25
ogain: 3.0
particles: 50
qos_overrides:
  /parameter_events:
    publisher:
      depth: 1000
      durability: volatile
      history: keep_last
      reliability: reliable
  /tf:
    publisher:
      depth: 100
      durability: volatile
      history: keep_last
      reliability: reliable
resampleThreshold: 0.5
sigma: 0.02
srr: 0.1
srt: 0.2
str: 0.1
stt: 0.2
temporalUpdate: 1.0
transform_publish_period: 0.05
use_sim_time: false
xmax: 10.0
xmin: -10.0
ymax: 10.0
ymin: -10.0
```

